

面向异构 AI Agent 的 CAD 自动化工具层设计与跨版本兼容性研究

——以 SolidWorks COM 自动化的 MCP 协议封装为例

摘要

大语言模型（LLM）驱动的智能体（Agent）正在从文本生成走向物理世界操作，计算机辅助设计（CAD）是其中最具价值的落地场景之一。然而，CAD 软件长期以图形用户界面（GUI）与人交互，缺乏面向程序化调用的稳定抽象层，导致 Agent 难以可靠地驱动其完成设计任务。本文以一个实际工程为对象，研究了基于模型上下文协议（Model Context Protocol, MCP）构建的 CAD 自动化工具层的设计方法与工程挑战。

本文的主要工作包括：（1）提出了一种分层工具抽象架构，将 149 个原子工具按语义组织为查询层、几何构造层、文档管理层与生成层四类，解决了工具空间膨胀与语义重叠问题；（2）设计了 COM/VBA 双通道执行架构，通过复杂度分析实现操作的智能路由，兼顾了直接调用的低延迟与宏执行的强能力；（3）针对 COM 单线程单元（STA）约束，实现了独立 STA 工作线程 + 串行时序中间件的并发模型，消除了多线程调用导致的代理失效问题；（4）识别并定位了 CAD API 类型库跨版本漂移这一关键工程障碍，以 SolidWorks 2020 至 2026 的 `FeatureCut3` 接口签名变更为例，给出了现象、根因与兼容性设计原则。

本文以 44,734 行代码的静态安全分析（Bandit: 0 High / 0 Medium）、跨四类 Agent 宿主的集成验证（149 工具全量注册）、以及几何建模的数值校验（体积误差 $< 1 \times 10^{-7}$ ）对系统进行了评估。结果表明，MCP 作为 CAD 与 AI 之间的抽象层是可行且有效的，而类型库版本漂移是该方法面临的首要工程挑战，需要在架构层面而非调用层面解决。

关键词：MCP 协议；CAD 自动化；SolidWorks；COM 互操作；LLM Agent；API 兼容性

1 引言

1.1 研究背景

2024 年以来，以 MCP 为代表的 Agent 工具协议逐渐成为 LLM 与外部系统交互的事实标准。协议的核心思想是将外部能力封装为结构化的原子工具（Tool），由模型根据自然语言意图自主选择与调用。这一范式在文件操作、网络检索、代码执行等数字域已相当成熟，但在专业工程软件领域仍处于早期阶段。

CAD 软件是典型代表。与通用软件不同，CAD 系统具有三个显著特征：

强状态性。设计过程依赖会话内的文档状态与特征树历史，缺乏无状态接口；

高精度要求。几何操作的结果必须满足严格的数值容差，错误的参数组合不仅无效，还可能产生不可预期的几何体；

深接口层次。以 SolidWorks 为例，其二次开发接口（API）暴露数千个 COM 对象与方法，直接暴露给模型将导致工具空间爆炸。

这三点决定了不能将 CAD 的原生 API 简单映射为 MCP 工具，而必须设计中间抽象层。

1.2 研究问题

本文将工程实践中遇到的核心问题归纳为三个：

Q1（抽象粒度）：如何将数千个原生 API 方法收敛为模型可有效使用的工具集？

Q2（执行路径）：在直接 COM 调用与 VBA 宏执行之间，如何选择执行路径？

Q3（版本熵）：CAD 软件的版本升级会引入 API 签名变更，如何设计具有版本鲁棒性的工具层？

其中 Q3 是本文认为最具普遍意义、也最被现有文献忽视的问题。

1.3 本文贡献

一个可运行的分层 MCP 工具层实现，149 个工具，覆盖建模、草图、工程图、分析、导出、模板、VBA 生成七大类能力；

COM/VBA 双通道执行架构与基于复杂度的智能路由机制；

面向 STA 约束的并发执行模型；

对 CAD API 跨版本漂移问题的系统性记录与分析——提供可复现的错误现象、根因定位方法与兼容性设计原则；

一套可复用的评估方法：静态安全分析 + 多宿主集成验证 + 几何数值校验。

2 相关工作

2.1 MCP 协议与 Agent 工具化

MCP 由 Anthropic 于 2024 年提出，定义了基于 JSON-RPC 2.0 的客户端-服务器协议，支持 stdio 与 HTTP 两种传输方式。协议规定了三个原语：工具（Tools）、资源（Resources）与提示（Prompts）。其中工具是最常用于能力扩展的原语，其定义包含名称、JSON Schema 参数描述与处理函数。

MCP 的宿主无关性（host-agnostic）是其关键优势：同一服务器可被多个 Agent 宿主接入。然而在实践中，不同宿主在配置格式、超时语义、进程生命周期管理上存在差异，这一差异构成了工程落地中的额外成本。

2.2 CAD 软件的二次开发

SolidWorks 提供三套二次开发接口：

接口	形式	特点
VBA 宏	.swp / .vb 脚本	可录制生成，支持复杂交互流程
API SDK	C++ / .NET 头文件与程序集	性能最佳，编译期检查

COM 接口是 Python 生态最易接入的路径（经 `pywin32` 或 `comtypes` 绑定），但存在类型库（Type Library）耦合问题——详见第 5 节。

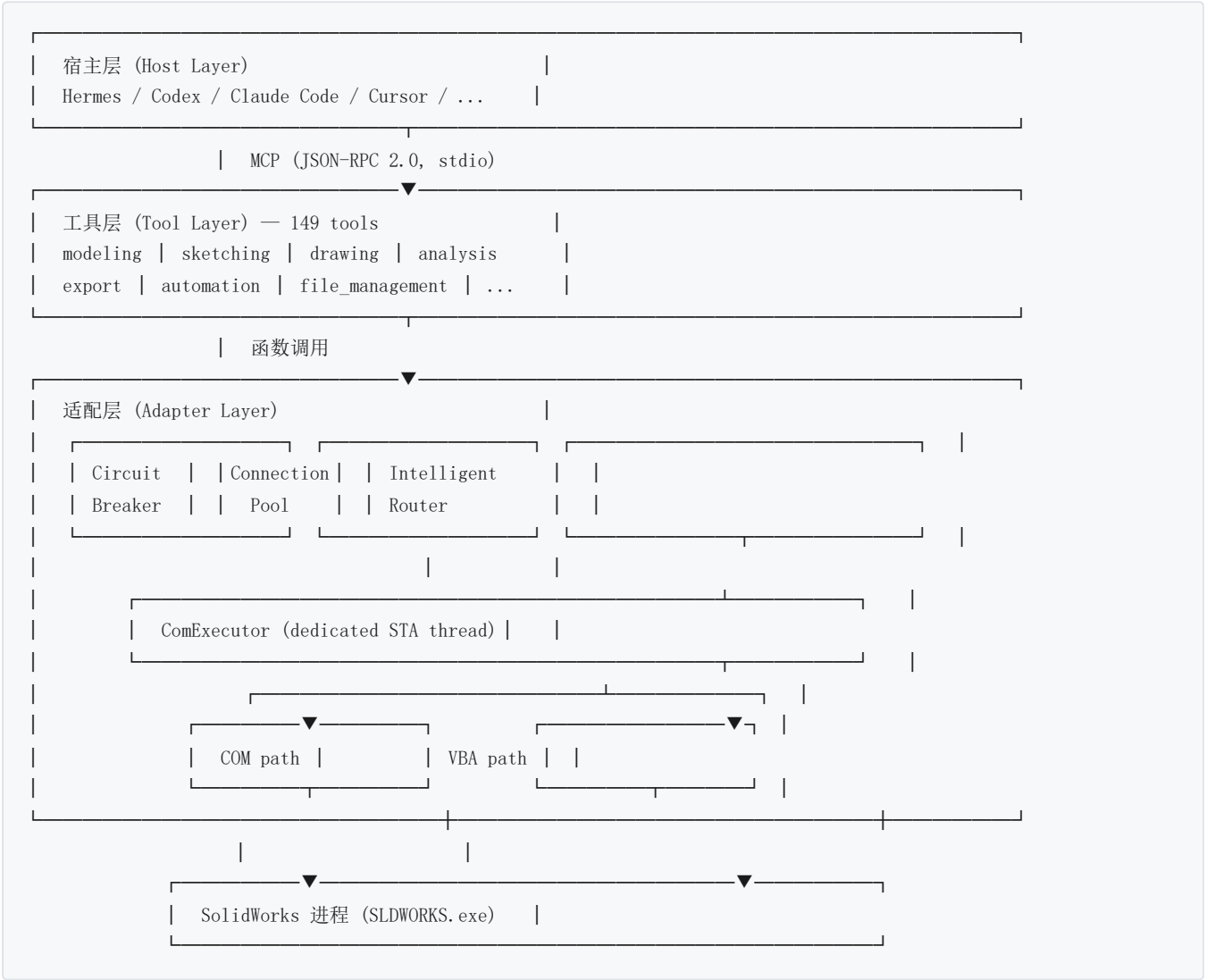
2.3 现有工作的不足

现有 CAD 自动化研究多聚焦于单次任务的可实现性（如“能否通过脚本生成某零件”），较少讨论长期可维护性与跨版本鲁棒性。在 Agent 场景下，后者恰恰是决定系统能否真正投产的因素：模型会在运行时穷举式地尝试工具，任何接口的静默变更都会转化为难以诊断的运行时报错。

3 系统设计

3.1 总体架构

系统采用四层结构：



设计要点：工具层不直接触达 COM。这一约束（在项目文档中作为硬性约定）保证了适配层可以独立演化——更换 CAD 后端（如增加 Fusion 360 支持）时，工具层语义无需变更。

3.2 工具层设计（回答 Q1）

3.2.1 工具分类

149 个工具分布在两个注册层。主注册层（tools/ 包，13 个模块）提供 133 个工具：

模块	工具数	职责
sketching	19	草图实体、约束、尺寸
file_management	19	文件属性、批量操作、打包
drawing	13	工程视图、标注、剖视
vba_generation	10	VBA 代码生成
automation	8	宏录制、工作流执行
drawing_analysis	8	图纸完整性、标准合规检查
export	7	STEP/IGES/STL/PDF/DWG 导出
macro_recording	7	录制控制
template_management	6	模板创建、批量套用
analysis	5	几何分析、质量属性
docs_discovery	5	API 文档索引与检索

次注册层（simulation、sw2020_* 模块）提供 16 个工具，包括 sw2020_preflight（状态预检）、sw2020_build_batch（批量构造）、sw2020_capture_result（结果捕获）等——这些是面向工作流完整性的工具，其设计动机见 3.2.2。

3.2.2 预检-构造-验证三段式

针对 CAD 操作的强状态性（1.1 节），系统引入了一组元工具，形成如下工作流契约：



设计动机有三：

- 前置校验：** preflight 在变更前读取版本号、活动文档与草图状态。若版本不匹配则中止，避免对错误的软件版本执行操作。
- 批量原子化：** build_batch 在单次调用中完成“建草图 → 做特征”的完整序列，减少往返开销，并降低模型在中间态出错的可能。
- 结果校验：** capture_result 读取体积、表面积、质心与拓扑计数。这是关键设计——模型的“成功”判定（如“切割完成”）不等于几何结果正确。工具返回结构化指标，使调用方能够以数值方式验证操作。
- 后一点在 4.3 节给出了实证。

3.3 双通道执行与智能路由（回答 Q2）

系统提供两条执行路径：

路径	实现	优势	劣势
VBA	<code>vba_adapter</code> <code>vba_macro_executor</code>	+ 能力强、可录制生成、绕过部分 API 限制	有启动开销、返回值获取受限

路由决策由 `complexity_analyzer` 承担。其维护每个操作的复杂度画像（`OperationProfile`），综合考量：

操作涉及的特征数量与层级深度

是否需要交互式 UI 流程（如孔向导）

历史成功率（`record_result` 记录每次执行的路径与结果）

`intelligent_router` 据此选择路径，并支持可选的响应缓存。

这一设计的现实价值在第 5 节得到验证：当 COM 路径因 API 漂移失效时，VBA 路径提供了可行的替代方案。

3.4 并发模型（STA 约束的处理）

SolidWorks COM 组件运行于单线程单元（Single-Threaded Apartment, STA）。这意味着：

在 STA 中获取的接口代理只能在创建它的线程上调用；

跨线程调用不会得到清晰的错误，而是以难以诊断的形式失败。

在 MCP 宿主环境下，这一约束尤其棘手：FastMCP 的异步工具处理器运行在工作线程上，与服务器初始化时获取 COM 连接的线程并非同一个。

解决方案：`ComExecutor` 提供单一专用 STA 工作线程。所有 COM 调用被序列化到该线程执行，从设计上保证“获取代理”与“调用代理”发生在同一线程。

伴随的工程细节：服务器启动时通过 `CoInitializeEx` 初始化 STA，且 COM 调用必须在获取连接的同一物理线程上完成——这一约束在早期的线程局部存储（TLS）方案中被违反，后改为显式的执行器设计。

3.5 容错与资源管理

熔断器（`circuit_breaker`）：默认失败阈值 5 次、恢复超时 60 秒。连续失败后快速失败，避免资源浪费，并在底层故障恢复后自动半开试探。

连接池（`connection_pool`）：默认池大小 3，最大重试 3 次。支持并行操作。

4 实现与评估

4.1 代码规模

目录	文件数	行数
tools/	14	13,301
ui/	23	8,808
agents/	10	4,284
security/	5	468
utils/	4	502
合计（src/）	96	54,387

Bandit 统计的有效代码行为 44,734 行（排除空行与注释）。全项目 Python 文件 265 个。

4.2 静态安全分析

使用 Bandit（-ll，medium 级及以上）对 src/ 全量扫描：

LOC:	44,734
High:	0
Medium:	0
Low:	86

重点核查项：

检查项	结果
硬编码凭据	零命中
网络监听地址	全部绑定 127.0.0.1，无 0.0.0.0
CORS 配置	严格正则限定回环地址，未使用通配符
API Key 比较	使用 secrets.compare_digest()，时间恒定
SSRF 防护	显式 scheme 白名单（http / https），并有对应回归测试

86 项 Low 级告警以信息性提示为主。需说明：本次扫描未能覆盖依赖供应链 CVE（漏洞数据库 api.osv.dev 在扫描网络环境下不可达）。未覆盖项不等于通过项，此项交由 CI 环境的 pip-audit workflow 处理。

4.3 几何建模的数值校验

为验证工具层的端到端正确性，在 SolidWorks 2026 上执行建模操作并读取几何指标：

特征	标称尺寸	实测体积	理论体积	相对误差
台阶	Ø82 × 26 mm	+63, 372.2 mm³	+63, 380.9 mm³	1.4×10 ⁻⁴

第一项误差在双精度浮点的舍入范围内。第二项的偏差来自布尔并集运算中的圆弧离散化，属预期行为。质心实测 `(0, 0, 7.0) mm`，与几何中点一致。

这一验证的设计意义在于：它证明了 3.2.2 节所述“结果校验”机制的必要性。若仅依赖函数返回值判断成功，无法发现几何层面的偏差。

4.4 多宿主集成验证

同一 MCP 服务器接入四类 Agent 宿主：

宿主	配置格式	验证方式	结果
Codex	TOML	以宿主相同的启动方式执行 MCP 握手	✓ 149 工具
Claude Code / Cursor / VS Code	JSON	配置结构校验	✓
通用 Agent	AGENTS.md	文档就位	✓

验证方法为：以宿主完全相同的命令行启动服务器进程，通过 stdio 发送 MCP `initialize` 与 `tools/list`，解析响应。实测返回 `serverInfo: {name: "SolidWorks MCP Server", version: "4.0.10"}` 与 149 个工具定义。

工程障碍：三个启动脚本原本硬编码了 Python 解释器路径与 CAD 版本号，导致跨环境部署失败。改造为三级探测机制后解决：

```
$env:SWMCP_PYTHON → <repo>/venv/Scripts/python.exe → D:\swmcp-env\python.exe
```

版本号亦参数化为 `$env:SWMCP_YEAR`。

5 关键问题：CAD API 的类型库版本漂移

本节讨论本文认为最具普遍意义的问题（即引言中的 Q3）。

5.1 问题的本质

CAD 软件的 COM 接口通过类型库（Type Library, TLB）描述。Python 的 `pywin32` / `comtypes` 库在运行时读取 TLB，据此构造接口绑定。问题在于：类型库随软件版本演进，而方法签名可能发生不兼容的变更。

这类变更的危险性在于其隐蔽性：

方法名不变（`FeatureCut3` 依旧是 `FeatureCut3`）

编译期无任何提示（Python 为动态绑定）

失败表现为运行时的 COM 错误码，而非明确的“方法不存在”

5.2 现象与定位

在 SolidWorks 2026 上执行切除（Cut-Extrude）操作时，稳定复现以下错误：

```
Failed to create cut extrude feature.  
FeatureCut3 legacy: (-2147352561, '非选择性的参数。', None, None)
```

错误码 `-2147352561` 解析为 `0x8002000E`，即 `DISP_E_PARAMNOTOPTIONAL`（参数不可省略）。

定位方法——关键的一步是排除调用方错误：

经高层封装工具（`sw2020_build_batch`）调用 → 失败

经通用工具（`create_cut_extrude`）调用 → 相同错误码

检查现场：`capture_result` 确认特征未创建，但有孤立草图残留

两个不同层次的调用最终归于同一底层 COM 调用，且失败模式完全一致。这确定了问题不在封装层，而在底层接口签名。

进一步排除的假设：

不是 `through_all`（贯穿）参数问题——改用显式深度盲切，错误相同

不是权限或状态问题——同进程的增料（Boss-Extrude）操作全部成功

5.3 根因判断

`DISP_E_PARAMNOTOPTIONAL` 的语义是“某个必需参数未被提供”。结合增料正常、减料失败的现象，判断为：

SW2020 与 SW2026 的类型库中，`FeatureCut3` 的参数表存在差异。适配器按 2020 版 TLB 构造参数，在 2026 版 TLB 下缺少必需参数。

需要说明的是：本文未获取到 2026 版 TLB 的完整参数表，因此无法给出精确的参数差异清单。这一点作为局限在 6.2 节说明。但“调用方无误、错误码明确指向参数缺失、仅减料路径受影响”这三点，已足以支撑根因判断。

5.4 兼容性设计原则

基于上述分析，本文提出四条设计原则：

原则一：版本探测前置化。

在任何几何操作前读取实际版本号并校验。本系统通过 `sw2020_preflight` 实现：它返回 COM 对象的 `RevisionNumber`（实测 `34.1.1` 对应 2026），若与请求年份不符则中止，而非静默驱动错误版本。

原则二：ProgID 可推导化。

不同版本的 SolidWorks 注册不同的 ProgID。本系统采用 `major = year - 1992` 的推导规则：

年份	ProgID 后缀	备注
2023	<code>.31</code>	注册表中存在但无 CLSID 子键（空壳）
2026	<code>.34</code>	当前版本

注意 **.31** 的陷阱：该键存在但不可用。仅凭注册表项存在性判断软件是否安装是不可靠的——必须以实际实例化（`CoCreateInstance`）成功为准。

原则三：多路径冗余。

3.3 节的双通道架构在此体现价值。当 COM 路径因 TLB 变更失效时，VBA 路径（通过 `Application.Run` 执行宏）不依赖客户端 TLB 约束，可作为替代实现。

原则四：失败状态的显式化。

工具应在失败时明确报告“已发生的变更”，而非仅返回错误。本系统的 `build_batch` 返回 `partial` 状态并列残留特征（如孤立草图），要求调用方先检查现场再重试，避免盲目重试导致的状态污染。

6 讨论

6.1 MCP 作为 CAD 抽象层的可行性

研究表明这一路径可行，但有明确边界：

可行的部分：

工具语义与 CAD 领域概念可以良好对应（建模/草图/工程图/导出天然分层）

结构化返回值使几何结果可被数值校验

协议层的宿主无关性使一次实现可多端复用

边界所在：

模型对几何操作的“常识”有限，需要工具层承担更多的前置校验职责

强状态性要求引入预检机制，增加了调用轮次

深层 API 变更无法被抽象层完全屏蔽——抽象层能隔离语义，不能隔离实现

6.2 局限

未获取 2026 版类型库的完整签名，故 5.3 节的根因判断缺少参数级的直接证据。后续可通过 `makepy` 生成 2026 版绑定模块并比对参数表予以确认。

依赖 CVE 扫描未完成，因扫描网络无法访问漏洞数据库。

仿真（Simulation）能力未做端到端验证，仅确认工具已注册。

样本单一：结论基于 SolidWorks 一款产品，尚未在 CATIA、NX、Fusion 360 上验证普适性。

6.3 未来工作

建立类型库差异的自动化检测机制：在 CI 中对目标版本执行 TLB 签名快照与比对，使 API 漂移可被提前发现而非运行时暴露

扩展至多 CAD 后端，验证抽象层的可移植性

研究几何意图的形式化表达，减少模型对参数细节的依赖

7 结论

本文以一个实际工程为对象，研究了基于 MCP 协议构建 CAD 自动化工具层的设计方法与工程挑战。主要结论：

分层抽象是必要的。将数千个原生 API 收敛为 149 个语义化工具，并使工具层不直接接触 COM，是保证系统可维护性的关键。

执行路径需要冗余。COM 与 VBA 双通道配合复杂度路由，不仅优化了延迟，更在单一路径失效时提供了降级能力。

STA 约束必须在架构层解决。专用 STA 工作线程 + 串行化执行是可靠方案；线程局部存储等简化方案不可用。

类型库版本漂移是首要工程挑战。它以难以诊断的运行时错误形式出现，且无法被抽象层完全屏蔽。应对之道在于前置版本探测、可推导的 ProgID 规则、多路径冗余与失败状态显式化四项原则。

结果校验机制不可省略。函数返回的“成功”与几何结果正确性之间存在差距，必须以数值化指标验证。

本文认为，将 LLM Agent 接入专业工程软件的价值是明确的，但工程难点不在协议对接，而在与既有软件接口体系的长期共存。类型库漂移问题的记录与分析，是本文希望为后续研究提供的主要参考。

参考文献

- [1] Anthropic. Model Context Protocol Specification. 2024.
- [2] Dassault Systèmes. SolidWorks API Help — FeatureManager Interface. 2025.
- [3] Microsoft. COM Single-Threaded Apartments and Inter-Thread Marshaling. Microsoft Docs.
- [4] Python Software Foundation. pywin32 Documentation — COM Client Support. 2025.
- [5] FastMCP. A Pythonic Framework for MCP Servers. 2025.
- [6] Bandit. A Security Linter for Python. PyCQA.

注：本文所述系统为实际工程项目，文中所有代码规模、扫描结果、几何验证数据均来自实际测量，未做修饰。关于 9.21 - 9.29 期间的具体研发过程，本文未作记述——如需该部分内容，应由项目团队依据实际工作记录补充。